FinSight RAG

Project Requirements: a Question-Answering Assistant for Indian Company Financial Reports

Item	Detail
Owner	Vishwesh Patial
Goal	Portfolio project demonstrating AI Engineer skills: RAG, vector search, LLM evaluation, API design, and deployment
Budget	Rs 0 (free and open-source tools only)
Estimated effort	About 2-3 weeks part-time (roughly 30-40 hours)
Deliverables	Public GitHub repository, live demo link, evaluation results table, README with architecture diagram, 2-minute demo video

1. Project Overview

Problem. Annual reports and earnings call transcripts of listed Indian companies run to hundreds of pages. Analysts and retail investors spend hours searching them for specific facts such as revenue growth, margin commentary, risk factors, or management guidance.

Solution. FinSight RAG lets a user ask a question in plain English, for example *"What did the company say about operating margins in FY2025?"*, and returns a concise answer grounded in the documents, with citations to the exact company, document, and page number.

Why this project stands out. Most portfolio RAG apps stop at "it answers questions." This one adds a measured evaluation pipeline and documented experiments (chunking, reranking, hybrid search), which is what hiring managers look for in AI Engineer candidates.

2. Goals and Success Criteria

#	Success criterion	Target
G1	Ingest and index documents from at least 5 companies	5-10 companies, 10+ documents
G2	Answer questions with source citations (company, document, page)	100% of answers carry citations
G3	Retrieval quality on your golden test set	Hit rate@5 of 0.80 or higher
G4	Answer faithfulness (answer is supported by retrieved text)	0.85 or higher (RAGAS faithfulness)
G5	Refuse to answer when information is not in the documents	Correct refusal on 90%+ of out-of-scope questions
G6	Response time on the deployed demo	Under 10 seconds per query
G7	Publicly deployed and reproducible	Live demo link plus one-command Docker run

The targets are guides, not hard pass/fail lines. What matters most is that you measure honestly and show how each change moved the numbers.

3. Scope

In scope (MVP, must build)

- PDF ingestion pipeline with text cleaning, chunking, and metadata (company, year, document type, page).
- Embedding and indexing into a local vector database.
- Query pipeline: retrieve relevant chunks, build a grounded prompt, generate an answer with citations.
- FastAPI backend with documented endpoints.
- Streamlit web interface.
- Evaluation pipeline with a golden question set and metrics.
- Docker packaging, CI with GitHub Actions, and free cloud deployment.

Stretch goals (add after MVP works)

- Cross-encoder reranking and hybrid search (BM25 plus vectors), with before-and-after metrics.
- Metadata filters in the UI (choose company and year).
- Conversation memory for follow-up questions.
- Table extraction from PDFs for financial statements.
- Streaming responses in the UI.

Out of scope

- User accounts and authentication.
- Real-time stock prices or investment advice.
- Training or fine-tuning an LLM (that is Project 3).

4. Technology Stack (all free)

Layer	Tool	Notes
Language	Python 3.11	Use a virtual environment (venv or uv).
Orchestration	LangChain or LlamaIndex	Pick one and use it consistently. LangChain is more common in job postings.
PDF parsing	PyMuPDF (fitz) or pypdf	PyMuPDF is faster and keeps page numbers easily.
Embeddings	BAAI/bge-small-en-v1.5 via sentence-transformers	Runs on CPU locally and on free hosting.
Vector database	ChromaDB (persistent, local)	FAISS is an alternative. Chroma supports metadata filters out of the box.
LLM (local development)	Ollama with Llama 3.x 8B or Qwen 2.5 7B	Free and offline. Needs roughly 8-16 GB RAM.
LLM (deployed demo)	Google Gemini API or Groq API free tier	Free hosting cannot run Ollama, so use an API here. Check current free-tier limits.
Model switching	LiteLLM or LangChain chat model interface	Switch providers through one config value.
Reranker (stretch)	BAAI/bge-reranker-base (cross-encoder)	Free, runs on CPU.
Keyword search (stretch)	rank_bm25	For hybrid retrieval.
Backend API	FastAPI plus Uvicorn	Auto-generated Swagger docs at /docs.
Frontend	Streamlit	Calls the FastAPI backend.

Layer	Tool	Notes
Evaluation	RAGAS plus your own retrieval metrics	RAGAS uses an LLM as judge. Use the Gemini or Groq free tier.
Testing and linting	pytest, ruff	Run in CI.
Packaging	Docker, docker-compose	One command to run the whole app.
CI/CD	GitHub Actions	Free for public repositories.
Hosting	Hugging Face Spaces (Docker or Streamlit SDK)	Free CPU tier. Store API keys as Space secrets.

5. Data Requirements

- **Sources:** annual reports (integrated reports) and earnings call transcripts from company investor-relations pages, NSE, or BSE. They are publicly available as PDFs.
- **Suggested companies (pick 5-10 across sectors):** TCS, Infosys, HDFC Bank, ICICI Bank, Reliance Industries, Asian Paints, Maruti Suzuki, ITC, Sun Pharma, Larsen and Toubro.
- **Volume:** start with 1 annual report per company for the latest year, then add the previous year to allow year-over-year questions.
- **Storage:** keep PDFs in `data/raw/` and add that folder to `.gitignore`. Annual reports are large, so provide a `scripts/download_data.py` script or a list of source links instead of committing the files.
- **Metadata to capture per chunk:** company, ticker, fiscal year, document type (annual report or transcript), source file name, page number, chunk id.

6. System Architecture

6.1 Ingestion pipeline (offline, run once per new document)

```
PDF files (data/raw/)
   |
   v
[1] Load PDF page by page (PyMuPDF)  -> keep page numbers
   |
   v
[2] Clean text: remove headers/footers, page numbers, extra whitespace
   |
   v
[3] Chunk: RecursiveCharacterTextSplitter (e.g. 800 chars, 150 overlap)
   |        attach metadata {company, year, doc_type, page, chunk_id}
   v
[4] Embed chunks (bge-small-en-v1.5)
   |
   v
[5] Store vectors + text + metadata in ChromaDB (data/index/)
```

6.2 Query pipeline (online, per user question)

```
User question (+ optional company/year filter)
   |
   v
[1] Embed question with the same embedding model
   |
   v
[2] Retrieve top-k chunks from ChromaDB (k = 8), apply metadata filter
   |
   v
[3] (Stretch) Hybrid: merge with BM25 results; rerank with cross-encoder -> top 4
   |
   v
[4] Build grounded prompt: system rules + numbered context chunks + question
   |
   v
[5] LLM generates answer citing chunk numbers, or says "not found"
   |
   v
[6] Return {answer, sources[company, doc, page, snippet], latency_ms}
```

6.3 Prompt requirements

- Answer only from the provided context. If the answer is not in the context, reply that the information was not found in the documents.
- Cite sources inline using the chunk numbers, e.g. [1], [3], and map them to company, document, and page in the response.
- Keep numbers exactly as written in the source, including units (crore, lakh, %).
- Keep answers concise: 2-6 sentences unless the user asks for detail.

7. Functional Requirements

ID	Requirement	Priority
FR1	Ingest one or more PDFs from a folder via a CLI command (python -m finsight.ingest --path data/raw).	Must
FR2	Skip documents already indexed (check by file hash) so ingestion can be re-run safely.	Must
FR3	Store each chunk with the full metadata listed in Section 5.	Must
FR4	Accept a natural-language question and return an answer with citations.	Must
FR5	Allow optional filtering by company and fiscal year.	Must
FR6	Return a clear "not found in documents" response for out-of-scope questions.	Must
FR7	Show the retrieved source snippets and page numbers in the UI.	Must
FR8	Make chunk size, overlap, top-k, embedding model, and LLM configurable through a config file or .env.	Must
FR9	Log each query with latency, retrieved chunk ids, and model used.	Should
FR10	Support hybrid retrieval and reranking behind a config flag.	Stretch
FR11	Support follow-up questions using chat history.	Stretch

8. Non-Functional Requirements

- **Security:** never commit API keys. Use a `.env` file locally (listed in .gitignore) and provide `.env.example`. Use Space secrets when deployed.

- **Performance:** under 10 seconds per query on the free deployed tier; ingestion of one 300-page report in under 5 minutes on a laptop.
- **Reproducibility:** pinned dependencies in `requirements.txt` or `pyproject.toml`; `docker compose up` starts everything.
- **Code quality:** type hints, docstrings on public functions, ruff passes with no errors.
- **Testing:** unit tests for chunking, metadata, and the API; at least 60% coverage of the core package.
- **Error handling:** meaningful API error messages for empty questions, missing index, and LLM timeouts, with a retry for transient LLM errors.

9. API Specification

Method	Endpoint	Purpose
GET	/health	Returns status and number of indexed chunks.
POST	/query	Answers a question with citations.
GET	/documents	Lists indexed documents with company, year, and chunk count.
POST	/ingest (optional)	Triggers ingestion of files in data/raw. Disable in the public demo.

Example request and response for POST /query

```
// Request
{
  "question": "What were the main risk factors mentioned by Infosys?",
  "company": "Infosys",        // optional
  "fiscal_year": "FY2025",     // optional
  "top_k": 8                   // optional
}

// Response
{
  "answer": "Infosys highlighted ... [1][3]",
  "sources": [
    {"id": 1, "company": "Infosys", "document": "Annual Report FY2025",
     "page": 142, "snippet": "..."},
    {"id": 3, "company": "Infosys", "document": "Annual Report FY2025",
     "page": 145, "snippet": "..."}
  ],
  "model": "gemini-...",
  "latency_ms": 3120
}
```

10. Evaluation Plan (the most important section)

10.1 Golden test set

- Create `eval/golden_set.jsonl` with 50 questions written by you after reading parts of the reports.
- Each entry holds: question, expected answer, source company, source page(s).
- Mix of question types: 20 factual lookups (numbers), 15 explanation questions ("why did margins fall"), 10 comparison or multi-part questions, and 5 out-of-scope questions whose correct response is "not found."

```
{"id": "q01", "question": "What was TCS's revenue in FY2025?",
 "expected_answer": "...", "company": "TCS", "pages": [12], "type": "factual"}
```

10.2 Metrics

Metric	What it measures	How
Hit rate@k	Whether a correct page appears in the top k retrieved chunks	Your own script, compare retrieved pages to golden pages
MRR	How high the first correct chunk is ranked	Your own script
Faithfulness	Whether the answer is supported by the retrieved context	RAGAS
Answer relevancy	Whether the answer addresses the question	RAGAS
Refusal accuracy	Correct "not found" on out-of-scope questions	Your own script
Latency (p50, p95)	Response time	Logged per query

10.3 Experiments to run and report

Exp	Configuration	What you learn
E1	Baseline: chunk 800 / overlap 150, top-k 4, vector search only	Reference numbers
E2	Chunk size 400 vs 800 vs 1200	Effect of chunk size on retrieval
E3	Top-k 4 vs 8	Recall vs noise trade-off
E4	E1 plus cross-encoder reranking	Value of reranking
E5	E1 plus hybrid BM25 + vector	Value of keyword matching for numbers and names
E6	Local Ollama model vs free-tier API model	Quality, speed, and cost trade-off

Put the results in a table in your README, with one short paragraph explaining which configuration you chose for the deployed demo and why. This table is the part interviewers will ask about.

11. Repository Structure

```
finsight-rag/
|-- README.md
|-- pyproject.toml / requirements.txt
|-- .env.example
|-- .gitignore
|-- Dockerfile
|-- docker-compose.yml
|-- config.yaml                # chunk size, top_k, models
|-- data/
|   |-- raw/                   # PDFs (gitignored)
|   `-- index/                 # Chroma DB (gitignored)
|-- scripts/
|   `-- download_data.py
|-- src/finsight/
|   |-- __init__.py
|   |-- config.py
|   |-- ingest.py              # load, clean, chunk, embed, store
|   |-- retriever.py           # vector, hybrid, rerank
|   |-- prompts.py
|   |-- generator.py           # LLM call + citation mapping
|   |-- pipeline.py            # end-to-end query function
|   `-- api.py                 # FastAPI app
|-- app/
|   `-- streamlit_app.py
|-- eval/
|   |-- golden_set.jsonl
|   |-- run_eval.py
|   `-- results/               # CSV/JSON results per experiment
```

```
|-- tests/
|   |-- test_chunking.py
|   |-- test_retriever.py
|   `-- test_api.py
|-- docs/
|   `-- architecture.png
`-- .github/workflows/ci.yml    # ruff + pytest on every push
```

12. Milestones and Timeline

Phase	Tasks	Done when	Time
1. Setup	Create repo, virtual environment, folder structure, .gitignore, config file. Download 5 reports.	Repo pushed to GitHub with structure and README stub	Days 1-2
2. Ingestion	PDF loading, cleaning, chunking with metadata, embeddings, Chroma storage, re-run safety.	All documents indexed; chunk counts printed	Days 3-5
3. Query pipeline	Retriever, prompt, LLM call via Ollama, citation mapping, not-found handling.	Correct cited answers in a notebook or CLI	Days 6-8
4. API and UI	FastAPI endpoints, Streamlit UI with sources panel and filters.	App works end to end locally	Days 9-11
5. Evaluation	Write 50 golden questions, eval script, run E1-E3 (then E4-E6 if time).	Results table filled in	Days 12-15
6. Ship	Tests, ruff, Dockerfile, GitHub Actions CI, deploy to Hugging Face Spaces, README, demo video.	Live link works; CI badge is green	Days 16-20

13. Deployment Steps

- Write a `Dockerfile` based on `python:3.11-slim` that installs dependencies and starts the app.
- Run locally with `docker compose up` and confirm the UI and `/docs` both work.
- Pre-build the Chroma index locally and include it in the Space (or build it at startup if small), so the demo does not ingest on every launch.
- Create a Hugging Face Space (Docker SDK), push the code, and add your Gemini or Groq API key under Space Settings as a secret.
- Set the LLM config to the API provider for the deployed version, since Ollama cannot run on the free Space.
- Add `.github/workflows/ci.yml` to run ruff and pytest on every push, and add the CI badge to the README.
- Test the live link on your phone and in an incognito window before sharing it.

14. README Checklist

The README is what recruiters actually read. It should include, in this order:

- Project title, one-line description, live demo link, and CI badge at the top.
- A GIF or 2-minute demo video.
- The problem and what the app does (3-4 sentences).
- Architecture diagram (draw it in draw.io or Excalidraw, both free).
- Tech stack list.
- Evaluation results table and key findings (for example: "Reranking improved hit rate@5 from 0.72 to 0.86").
- How to run locally: clone, set .env, docker compose up.

- Design decisions and trade-offs (why this chunk size, why this model).
- Limitations and future work.

15. Resume Entry Template

Fill in the brackets with your real results after running the evaluation. Never put a number on your resume that you did not measure.

```
FinSight RAG: Financial Report Q&A Assistant | Python, LangChain, ChromaDB,
FastAPI, Docker, Hugging Face Spaces | github.com/BugHuntre/finsight-rag

- Built a retrieval-augmented generation (RAG) system answering questions over
  [N]+ annual reports of [N] Indian listed companies ([N]k+ chunks), returning
  page-level citations, deployed via FastAPI, Streamlit, and Docker.
- Designed an evaluation pipeline on a [50]-question golden set using RAGAS;
  cross-encoder reranking and hybrid BM25 search raised retrieval hit rate@5
  from [X] to [Y] and faithfulness to [Z].
```

16. Questions Interviewers Will Ask About This Project

- Why did you choose this chunk size and overlap? What happened when you changed them?
- How do you stop the model from hallucinating, and how did you measure it?
- Why use a reranker? What does a cross-encoder do differently from an embedding model?
- Why did hybrid search help (or not help) with financial numbers and company names?
- How would you scale this to 10,000 documents? (Hints: managed vector DB, batching, caching, async.)
- How would you handle tables in PDFs?
- What would you change to run this in production for real users?

Write short answers to each of these in a `docs/notes.md` file while you build. It doubles as interview preparation.